Understanding Python Closures

A Prelude to Decorators

Functions behave as first-class objects in the Python runtime

- **First-class status** means functions can be treated like any other variable or data type.
- **Assign functions to variables** , pass them as arguments to other functions, and return them from other functions.
- **Nested definitions** allow an inner function to be defined entirely within the execution block of an outer function.
- **Nesting behavior** forms the structural foundation of scope-sharing and environment capture.

1_first_class_functions.py

```python
def shout(text):
    return text.upper()

# Assign to a variable
yell = shout
print(yell('hello'))  # Outputs: 'HELLO'

def greet(func):
    # Pass function as an argument
    print(func('hi there'))

greet(shout)  # Outputs: 'HI THERE'
```

DESIGN INSIGHT: Key Principle

In Python, parenthesis () execute a function. Passing the name without parenthesis passes the actual function object reference.

Nested scopes resolve variables dynamically via LEGB lookup

- **LEGB rule** defines how Python looks up variable names in hierarchical order:
- **Local (L)** : Inside the current active function block.
- **Enclosing (E)** : Local scopes of parent or outer functions.
- **Global (G)** : Variables defined at the top level of the module.
- **Built-in (B)** : Pre-assigned names in the Python builtins module.
- **Enclosing scope (E)** is the critical zone for closures. Inner functions can natively read parent variables.

2_legb_scope_lookup.py

```python
def outer():
    # Enclosing scope variable
    message = 'Hi'

    def inner():
        # Local scope accessing enclosing scope
        print(message)

    inner()

outer()  # Outputs: 'Hi'
```

DESIGN INSIGHT: Lookups are Read-Only By Default

An inner function can freely READ any variable in its enclosing parent scopes, but cannot MODIFY it without using nonlocal.

Inner functions capture enclosing state to form a closure

- A closure is a nested function that retains access to variables in its parent scope even after the parent function has finished executing.
- The environment is packed with the returned function, keeping enclosing variables alive in memory rather than allowing them to be discarded when the parent scope closes.
- Local variables are normally destroyed once a function exits, but closures override this behavior by preserving the variables a returned function still needs.
- This mechanism enables function factories: outer functions that generate customized, independent function objects, each carrying its own captured state.

3_closure_basics.py

```
def make_multiplier(factor):
    # This variable is 'captured'
    def multiplier(number):
        return number * factor
    return multiplier

# Generate dynamic multiplier functions
double = make_multiplier(2)
triple = make_multiplier(3)

print(double(10))  # Outputs: 20
print(triple(10))  # Outputs: 30
```

DESIGN INSIGHT: Why did 'factor' survive?

When make_multiplier(2) completes, its scope would normally close. However, because 'double' holds a reference to the inner function, Python creates a permanent closure binding '2'.

Python stores enclosed environment data inside the __closure__ attribute

- Closures are **not magic** ; they are exposed transparently inside the Python object architecture.
- The **__closure__** attribute is a tuple containing cell objects, each corresponding to a captured variable.
- A **cell object** acts as an indirect storage cell keeping the variable's value alive.
- Each cell exposes its value through the **cell_contents** property.
- A function that is **not a closure** has its __closure__ attribute set to None.

4_closure_attribute_inspection.py

```
def make_multiplier(factor):
    def multiplier(number):
        return number * factor
    return multiplier

double = make_multiplier(2)

# Inspect the internal closure cells
print(double.__closure__)
# Outputs: (<cell at 0x...: int object at 0x...>,)

# Inspect cell contents
cell = double.__closure__[0]
print(cell.cell_contents)  # Outputs: 2
```

DESIGN INSIGHT: Under the Hood

The __closure__ attribute proves that closures are first-class namespace dictionaries linked to returned functions, maintaining their own isolated states.

The nonlocal statement modifies read-only parent variables

- Without the nonlocal keyword, assigning to a name inside a nested function creates a new local variable in that scope rather than modifying the variable in the enclosing scope.
- If a nested function reads a variable before it has been locally assigned, and no nonlocal declaration is present, Python raises an UnboundLocalError.
- The nonlocal keyword tells Python to search up through enclosing scopes and bind to that existing variable, rather than creating a new local one.
- This allows a closure to maintain modifiable state across multiple execution runs instead of resetting the value on each call.
- A common application is a private cache or counter, where state must persist between calls without being exposed as a global variable.

5_nonlocal_stateful_counter.py

```
def make_counter():
    count = 0
    def counter():
        nonlocal count  # Bind to outer scope
        count += 1
        return count
    return counter

my_counter = make_counter()
print(my_counter())  # Outputs: 1
print(my_counter())  # Outputs: 2
print(my_counter())  # Outputs: 3
```

DESIGN INSIGHT: Crucial Scope Difference

The nonlocal keyword only looks up to nested outer scopes. It will NOT look up to the Global module scope. Use the global keyword for module-level variables.

Stateful closures act as lightweight, low-overhead classes

When should you write a full OOP Class vs. generating a lightweight functional Closure?

Feature / Dimension	OOP Class Approach	Functional Closure Approach
State Encapsulation	Explicit using self attributes (self.x = value)	Implicit within enclosed environment scopes
Memory Footprint	Higher overhead due to class instance dictionary __dict__	Extremely lightweight, microsecond execution cost
Extensibility	Highly extensible via inheritance and multiple methods	Limited. Typically supports only a single callable behavior
Use Case sweet-spot	Complex objects with rich behavior, many actions	Simple helper function requiring lightweight state persistence

Closures vs Classes: If you need to bundle state with only one callable behavior, use a closure. If you need multiple methods or complex object lifecycles, write a class.

Why choose a closure over a plain function or a class?

The same task three ways: keep a running total that remembers every amount added so far.

1. PLAIN FUNCTION (GLOBAL STATE)

plain_function.py

```python
total = 0

def add(amount):
    global total
    total += amount
    return total

print(add(10))  # Outputs: 10
print(add(5))   # Outputs: 15
```

Works, but the total lives in a global variable: any code can change it, and the program can only hold one running total.

2. CLASS

class_version.py

```python
class Tally:
    def __init__(self):
        self.total = 0

    def add(self, amount):
        self.total += amount
        return self.total

shop = Tally()
print(shop.add(10))  # Outputs: 10
print(shop.add(5))   # Outputs: 15
```

Works, and each object keeps its own total, but a class, __init__, self and a method call are needed for just one behaviour.

3. CLOSURE

closure_version.py

```python
def make_tally():
    total = 0
    def add(amount):
        nonlocal total
        total += amount
        return total
    return add

shop = make_tally()
cafe = make_tally()
print(shop(10))  # Outputs: 10
print(shop(5))   # Outputs: 15
print(cafe(3))   # Outputs: 3
```

Private state in a few lines: each call to make_tally() returns a new, independent total that is called like an ordinary function.

DESIGN INSIGHT: Choosing Between the Three

Use a plain function when no state needs to be remembered between calls. Use a closure when a small piece of private state serves a single behaviour, such as a counter, a callback or a configured helper. Use a class when several methods share the state.

Function decorators leverage closure mechanics to wrap behaviors

- Decorators are design wrappers that take an input function, extend its behavior, and return a new function.
- The returned wrapper function acts as a closure, maintaining an active reference to the original input function even after the decorator function itself has finished executing.
- This structure allows decorators to inject common cross-cutting logic, such as logging, timing, or security checks, cleanly across a codebase without modifying each target function individually.
- The inner function wraps arguments using *args and **kwargs, allowing it to intercept and forward parameters dynamically regardless of the original function's signature.

6_decorator_via_closure.py

```
def logged(func):
    # Wrapper closure captures 'func'
    def wrapper(*args, **kwargs):
        print(f'Calling {func.__name__}')
        result = func(*args, **kwargs)
        print(f'Finished {func.__name__}')
        return result
    return wrapper

@logged
def add(a, b):
    return a + b

print(add(5, 7))
# Outputs:
# Calling add
# Finished add
# 12
```

DESIGN INSIGHT: Key takeaway

Closures are the foundational mechanics of decorators. Without the ability to 'close over' the original function parameter, decorators would not be able to execute or modify functional targets.